

E-HEALTH SASKATCHEWAMN

PATIENT LOOKUP SYSTEM

PRANJAL BAROT

1. Project Overview

The Patient Lookup System is a full-stack web application developed to securely manage and retrieve patient information.

The application consists of three main parts:

- **Frontend:** React with Vite
- **Backend:** Java Spring Boot REST API
- **Database:** PostgreSQL

The system also includes:

- User login
- JWT authentication
- Role-based authorization
- Patient CRUD operations
- Patient search
- Patient details
- Bootstrap and custom CSS
- Swagger/OpenAPI documentation
- Git/GitHub version control

The frontend communicates with the Spring Boot backend through REST APIs. The backend communicates with PostgreSQL to store and retrieve patient and user information.

2. Project Architecture

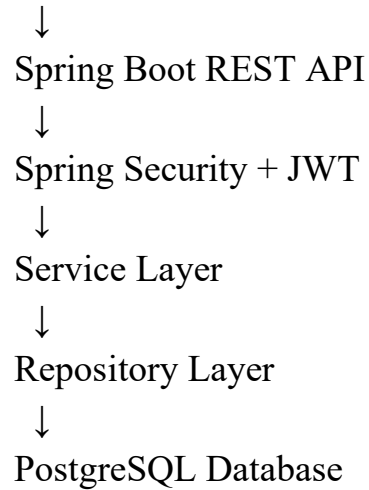
The application follows a three-layer architecture:

User



React Frontend

Patient Lookup System



Frontend

The React application provides the user interface. Users interact with pages such as Login, Patient List, Patient Details, and Search.

Backend

Spring Boot handles:

- API requests
- Authentication
- Authorization
- Business logic
- Patient CRUD operations
- Database communication

Database

PostgreSQL stores:

- User information
- Patient information
- Roles and other required data

3. Step 1 – Create the Project Folder

The project is organized into two major folders:

```
patient-lookup
├── backend
└── frontend
```

- The backend contains the Spring Boot application.
- The frontend contains the React/Vite application.
- This separation makes it easier to develop and maintain the frontend and backend independently.

4. Step 2 – Set Up the Backend

The backend is created using Spring Boot.

The main technologies used are:

- Java 21
- Spring Boot
- Spring Web
- Spring Data JPA
- Spring Security
- PostgreSQL Driver
- JWT library
- Maven

The backend is responsible for exposing REST APIs to the React application.

For example:

GET /api/patients: It can be used to retrieve patient records.

5. Step 3 – Configure PostgreSQL

PostgreSQL is used as the application's relational database.

A database is created for the project, for example:

patient_lookup

The Spring Boot application is then configured to connect to PostgreSQL.

Typical configuration contains:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/patient_lookup
spring.datasource.username=YOUR_USERNAME
spring.datasource.password=YOUR_PASSWORD
```

The database connection allows Spring Boot to save and retrieve patient information.

Important: Database passwords should not be committed to a public GitHub repository.

6. Step 4 – Create the Patient Entity

The Patient entity represents a patient record in the application.

The entity contains fields such as:

Patient ID, First Name, Last Name, Date of Birth, Phone, Email, Address, City, Province, Postal Code

The Patient class is mapped to a PostgreSQL table using JPA annotations.

The patient ID acts as the unique identifier for each patient.

7. Step 5 – Create the User Entity

A User entity is created to support authentication.

Typical fields include:

ID

Username

Password

Role

For example:

Username: admin/user

Password: Admin@123/User@123

Role: ADMIN/USER

The password should be stored securely using password hashing such as BCrypt rather than storing the original password.

8. Step 6 – Create Repository Classes

Spring Data JPA repositories are used to communicate with PostgreSQL.

For example:

```
public interface UserRepository extends JpaRepository<User, Long> {  
  
    Optional<User> findByUsername(String username);  
}
```

The repository automatically provides operations such as:

- Save
- Find
- Update

Patient Lookup System

- Delete
- Find by ID

For the Patient entity, a PatientRepository is created in the same way.

This avoids writing SQL manually for basic database operations.

9. Step 7 – Create the Service Layer

The service layer contains the application's business logic.

For patients, the service handles operations such as:

- Get all patients
- Get patient by ID
- Create patient
- Update patient
- Delete patient
- Search patients

The controller should not contain all of the business logic. Instead, it calls the service layer.

The general flow is:

Controller



Service



Repository



Database

10. Step 8 – Create the Patient Controller

The Patient Controller exposes REST API endpoints.

Typical endpoints include:

GET /api/patients

GET /api/patients/{id}

POST /api/patients

PUT /api/patients/{id}

DELETE /api/patients/{id}

GET: Retrieves patient information.

POST: Creates a new patient.

PUT: Updates an existing patient.

DELETE : Deletes a patient.

The controller receives HTTP requests from the React frontend and sends JSON responses back.

11. Step 9 – Implement Authentication

Authentication determines **who the user is**.

The application provides a Login page where the user enters:

Username

Password

- The React application sends these credentials to the backend.

For example:

POST /api/auth/login

- The backend verifies the credentials.
- If the credentials are valid, the backend generates a JWT token.

12. Step 10 – Generate JWT Token

JWT stands for **JSON Web Token**.

After successful login, the backend creates a token containing information such as:

Username

Role

Expiration time(SET THIS IN application.properties)

The token is signed by the backend.

The response may look conceptually like:

```
{  
  "token": "eyJhbGciOiJIUzI1NiJ9..."  
}
```

The React application stores the token and uses it for protected API requests.

13. Step 11 – Send JWT With API Requests

When the user requests protected information, the frontend sends the JWT using the HTTP Authorization header.

Example:

Authorization: Bearer <JWT_TOKEN>

- The backend receives the token and validates it.
- If the token is valid, the request can continue.
- If the token is missing or invalid, the backend rejects the request.

14. Step 12 – Create JWT Authentication Filter

A JWT authentication filter checks incoming requests.

The filter:

1. Reads the Authorization header.
2. Checks whether a Bearer token exists.
3. Extracts the JWT.
4. Validates the token.
5. Gets the username and role.
6. Creates an authenticated security context.
7. Allows the request to continue.

This allows Spring Security to recognize the logged-in user.

15. Step 13 – Implement Role-Based Authorization

The application uses roles such as:

USER

ADMIN

A normal user may be allowed to:

- View patients
- Search patients
- View patient details

An administrator can additionally be allowed to:

- Create patients

Patient Lookup System

- Update patients
- Delete patients

The exact permissions depend on the application's Spring Security configuration.

The important point is that authorization is enforced by the **backend**, not only by hiding buttons in React.

16. Step 14 – Create React Frontend

The frontend is created using React and Vite.

The frontend contains pages/components such as:

Login

Navbar

Patient List

Patient Details

Search

The frontend communicates with the Spring Boot backend using HTTP requests.

17. Step 15 – Configure React Router

React Router is used for navigation.

For example:

/login

/patients

/patients/1

/search

This allows the application to display different React components based on the current URL.

For example:

/login: displays the Login page.

/patients: displays the Patient List.

/patients/1 : displays the details for patient 1.

18. Step 16 – Create AuthContext

AuthContext manages authentication information throughout the React application.

It can keep track of:

- JWT token
- Login status
- User role
- Logout functionality

Instead of passing authentication information manually through every component, components can access it through the authentication context.

The application can also read the role from the JWT payload.

For example:

ADMIN

or

USER

19. Step 17 – Create Login Page

The Login page provides two input fields:

Username

Password

Patient Lookup System

The user enters their credentials and clicks the Login button.

The frontend sends the credentials to the authentication API.

If successful:

Login



JWT received



Token stored



User authenticated



Navigate to Patient List

If authentication fails, an error message is displayed.

20. Step 18 – Create Navbar

The Navbar provides navigation throughout the application.

It can contain links such as:

Home

Patients

Search

Logout

The Navbar can also change depending on whether the user is logged in.

For example, the Logout button should be available to an authenticated user.

When Logout is selected:

1. JWT is removed.
2. Authentication state is cleared.

3. User is redirected to Login.

21. Step 19 – Create Patient List Page

The Patient List page displays patient records retrieved from the backend.

The frontend sends:

GET /api/patients

The backend retrieves the data from PostgreSQL.

The response is returned as JSON.

React and then display the information.

22. Step 20 – Create Search Functionality

The Search page allows users to find patients more easily.

The user can enter information such as:

John

or

Smith

or a patient ID.

The application sends the search request to the backend or filters the retrieved data depending on the implemented design.

The results are displayed to the user.

Search functionality reduces the need to manually browse through a large patient list.

23. Step 21 – Create Patient Details Page

When a user selects a patient, the application navigates to the Patient Details page.

For example:

/patients/1001

The frontend sends:

GET /api/patients/1001

The backend retrieves patient 1001 from PostgreSQL and returns the information.

The React page then displays the complete patient details.

24. Step 22 – Implement Create Patient

Authorized users can create a new patient.

The user fills out a form such as:

First Name

Last Name

Date of Birth

Gender

Phone

Email

Address

The frontend sends a POST request:

POST /api/patients

The backend validates the data and saves it to PostgreSQL.

After successful creation, the application displays the new patient.

25. Step 23 – Implement Update Patient

- An authorized user can edit an existing patient.
- The application loads the existing information into a form.
- After changes are made, the frontend sends:
- PUT /api/patients/{id}
- The backend finds the patient, updates the information, and saves the changes.

26. Step 24 – Implement Delete Patient

- An authorized administrator can delete a patient.
- The frontend sends:
- DELETE /api/patients/{id}
- The backend verifies authorization and removes the patient from the database.
- A confirmation message should preferably be displayed before deletion because deletion is a destructive operation.

27. Step 25 – Add Validation

Validation ensures that incorrect data is not saved.

Examples include:

- Required first name
- Required last name
- Valid date of birth
- Valid email format
- Valid phone number
- Required username
- Required password

Validation should happen on both the frontend and backend.

Frontend validation improves user experience.

Backend validation provides the actual security boundary.

28. Step 26 – Handle Errors

The application should handle common errors properly.

For example:

401 Unauthorized: The user is not authenticated or the JWT is invalid.

403 Forbidden: The user is authenticated but does not have permission.

404 Not Found: The requested patient does not exist.

400 Bad Request: The submitted information is invalid.

500 Internal Server Error: An unexpected backend problem occurred.

The React application should display understandable messages rather than exposing technical errors to the user.

29. Step 28 – Connect Frontend and Backend

The React application needs to know where the Spring Boot API is running.

For example, if Spring Boot is running on:

<http://localhost:8080>

and React is running on:

<http://localhost:5173>

the React application sends API requests to the backend on port 8080.

The backend may also require CORS configuration so that requests from the React development server are allowed.

30. Step 29 – Run the Backend

Open the backend project in IntelliJ IDEA.

Make sure:

- Java 21 is configured.
- Maven dependencies are loaded.
- PostgreSQL is running.
- Database configuration is correct.

Then run the Spring Boot main application.

A successful startup should show that the application is running on the configured port, commonly:

8080

31. Step 30 – Run the Frontend

Open a terminal inside the frontend folder.

Run:

```
npm install
```

Then:

```
npm run dev
```

Vite will provide a local development URL, commonly:

<http://localhost:5173>

Open that address in the browser.

32. Step 31 – Test Login

Test with the configured test account.

Valid Login

Enter the correct username and password.

Expected result:

Login successful

JWT generated

Patient page displayed

Invalid Login

Enter an incorrect password.

Expected result:

Login failed

Error message displayed

33. Step 32 – Test Authorization

Authorization should be tested using different roles.

USER

Verify that the user can perform the operations permitted by the security configuration, such as:

View patients

Search patients

View patient details

Attempt an administrator-only operation and verify that the backend returns:

403 Forbidden

ADMIN

Verify that the administrator can perform the configured CRUD operations.

34. Step 33 – Test JWT

JWT testing should include:

No Token

Call a protected API without a token.

Expected:

401 Unauthorized

Invalid Token

Send an invalid token.

Expected:

401 Unauthorized

Expired Token

Send an expired token.

Expected:

401 Unauthorized

Valid Token

Send a valid JWT.

Expected:

Request accepted

35. Step 34 – Test CRUD

The complete CRUD process should be tested.

Create

Create a new patient and verify that it appears in PostgreSQL.

Read

Retrieve the patient through the API and frontend.

Update

Change patient information and verify the updated information.

Delete

Delete the patient and verify that it is no longer available.

36. Step 35 – Test Swagger/OpenAPI

Swagger/OpenAPI provides documentation and an interactive way to test the backend APIs.

It can be used to inspect:

- Login API
- Patient GET APIs
- Patient POST API
- Patient PUT API
- Patient DELETE API

- Request parameters
- Request bodies
- Response codes

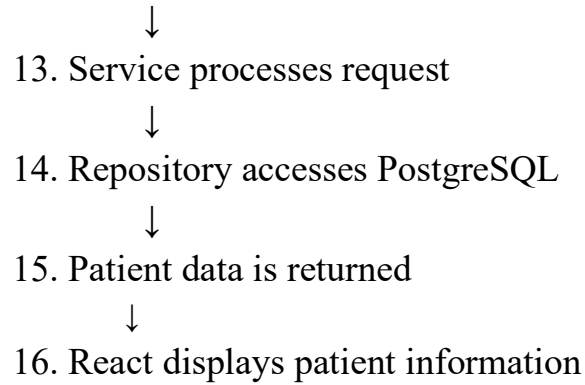
For JWT-protected endpoints, the token can be supplied through the Swagger authorization mechanism when configured.

37. Complete Application Flow

The complete system works like this:

1. User opens React application
↓
2. Login page appears
↓
3. User enters username/password
↓
4. React sends login request
↓
5. Spring Boot authenticates user
↓
6. JWT is generated
↓
7. React stores JWT
↓
8. User opens Patient List
↓
9. React sends JWT with API request
↓
10. Spring Security validates JWT
↓
11. Role is checked
↓
12. Controller receives request

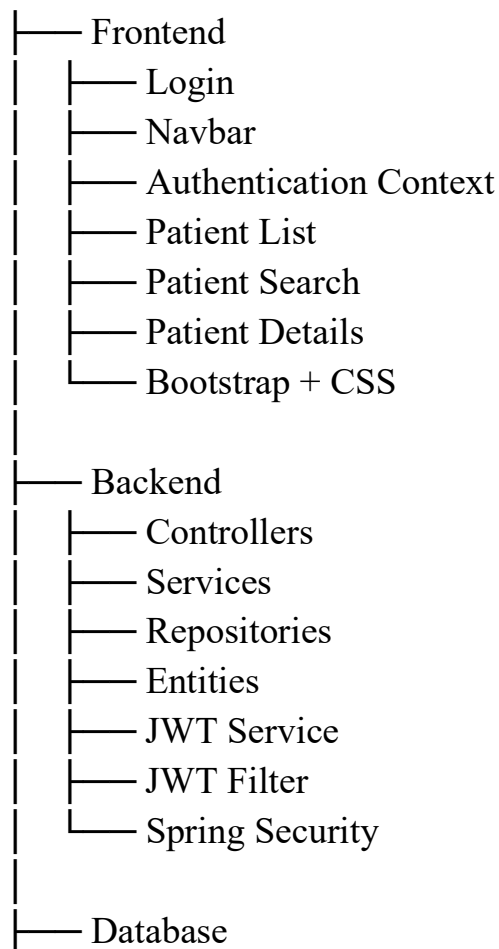
Patient Lookup System



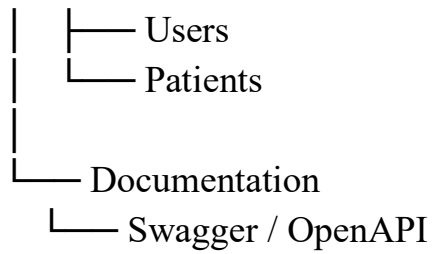
38. Final Project Structure

Your completed project can be understood as:

PATIENT LOOKUP SYSTEM

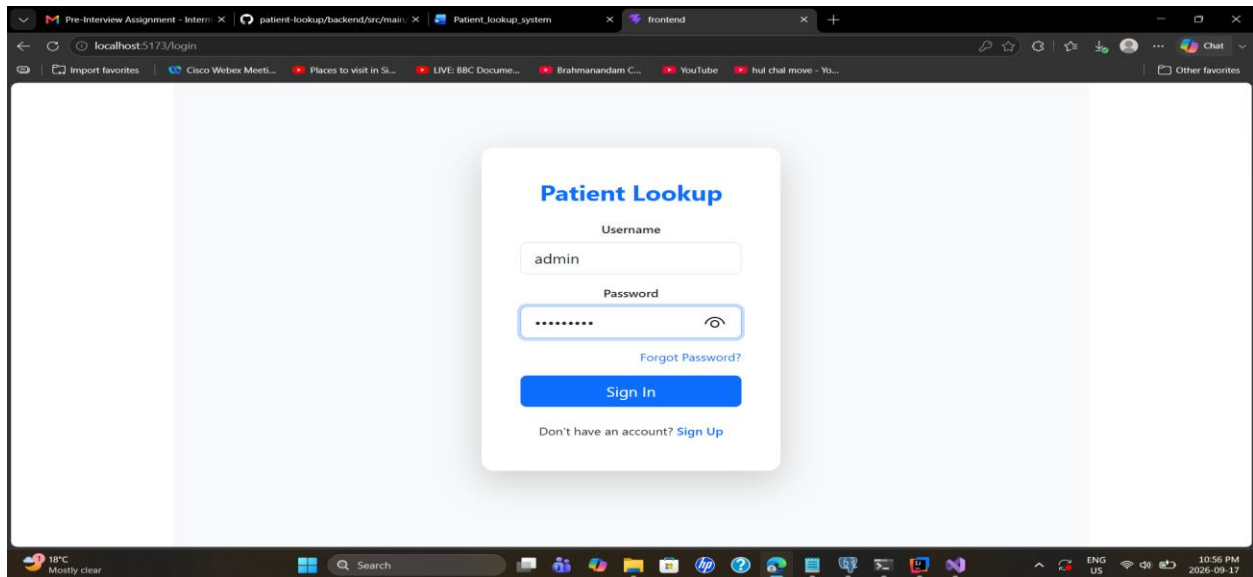


Patient Lookup System



39. Screenshots

Login page



Invalid username and password

Patient Lookup System

Patient Lookup

Invalid username or password.

Username

kmdemkl

Password

.....

[Forgot Password?](#)

Sign In

Don't have an account? [Sign Up](#)

Patients list

Patient Lookup

admin (ADMIN) [Logout](#)

Patients

Add Patient

Search by first or last name

Search

Clear

ID	First Name	Last Name	Date of Birth	Actions
2	pranjal	barot	2001-05-05	View Edit Delete
3	meet	patel	1997-09-28	View Edit Delete

Search option

Patients

Add Patient

pranjal

Search

Clear

ID	First Name	Last Name	Date of Birth	Actions
2	pranjal	barot	2001-05-05	View Edit Delete

Create patient page

Patient Lookup System

Patient Lookup

admin (ADMIN)

Logout

Create Patient

First Name			Last Name		
Date of Birth	yyyy-mm-dd		Phone		
Email			Address		
City	Province	Postal Code			
Create Patient		Cancel			

View option

Patient Lookup

admin (ADMIN)

Logout

Patient Details

pranjal barot

Patient ID: 2

Date of Birth: 2001-05-05

Phone: 6395542670

Email: pra@example.com

Address: rettalack street

City: regina

Province: SK

Postal Code: S4S 3B7

Update page

Patient Lookup

admin (ADMIN)

Logout

Update Patient

First Name	pranjal		Last Name	barot	
Date of Birth	2001-05-05		Phone	6395542670	
Email	pra@example.com		Address	rettalack street	
City	Province	Postal Code			
regina	SK	S4S 3B7			
Update Patient		Cancel			

Login as user (user doesn't authorize to delete patients)

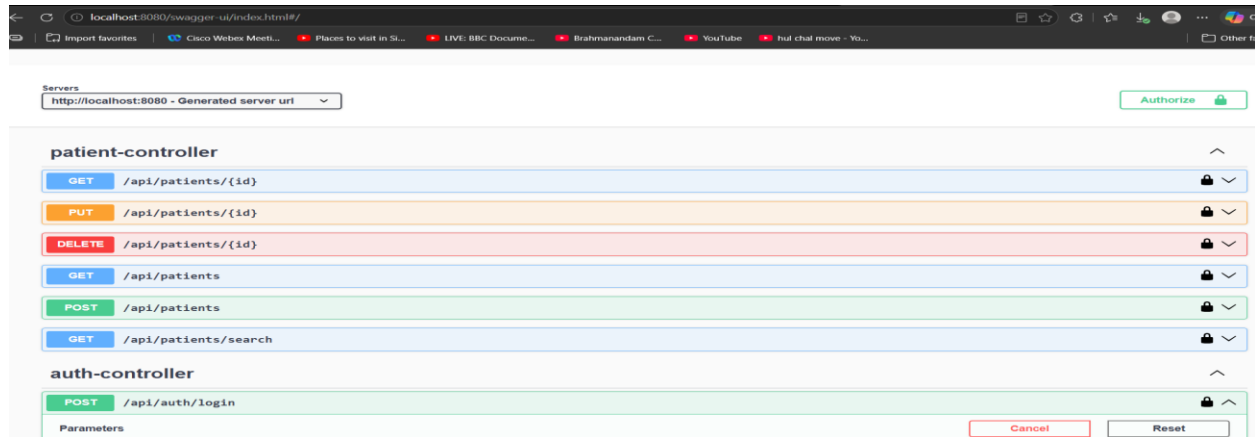
Patient Lookup System

Patients

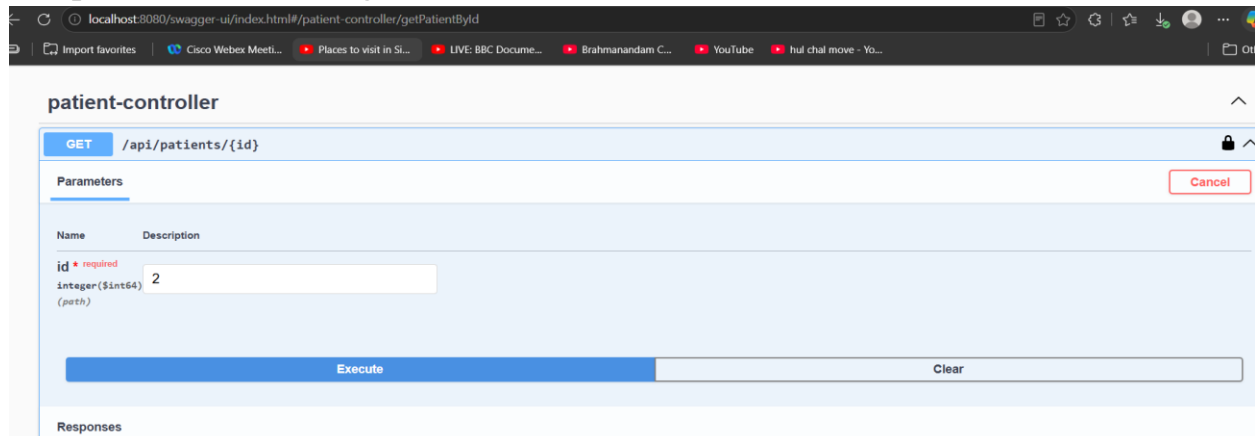
[Add Patient](#)[Search](#)[Clear](#)

ID	First Name	Last Name	Date of Birth	Actions
2	pranjal	barot	2001-05-05	View Edit
3	meet	patel	1997-09-28	View Edit

Swagger REST API testing

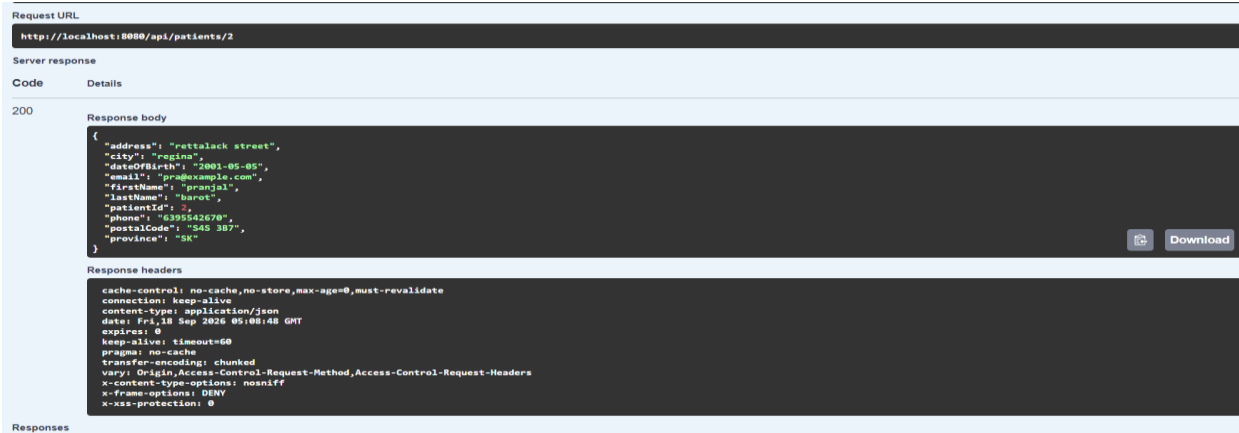


For example: GET patient by id
Get patient detail having id = 2



Request URL : <http://localhost:8080/api/patients/2>

Patient Lookup System



The screenshot shows a web browser interface for a REST client. At the top, the 'Request URL' is set to 'http://localhost:8080/api/patients/2'. Below this, the 'Server response' section shows a 'Code' of 200 and 'Details' for the response. The 'Response body' is a JSON object containing patient information: { "address": "rattalack street", "city": "Pegina", "dateOfBirth": "2001-05-05", "email": "prade@sample.com", "firstName": "pranjal", "lastName": "barot", "patientId": 2, "phone": "6395542670", "postalCode": "545 387", "province": "SK" }. The 'Response headers' section lists various headers including 'cache-control', 'connection', 'content-type', 'date', 'expires', 'keep-alive', 'pragma', 'transfer-encoding', 'vary', 'x-content-type-options', 'x-frame-options', and 'x-xss-protection'.

```
Request URL
http://localhost:8080/api/patients/2

Server response
Code    Details
200

Response body
{
  "address": "rattalack street",
  "city": "Pegina",
  "dateOfBirth": "2001-05-05",
  "email": "prade@sample.com",
  "firstName": "pranjal",
  "lastName": "barot",
  "patientId": 2,
  "phone": "6395542670",
  "postalCode": "545 387",
  "province": "SK"
}

Response headers
cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-type: application/json
date: Fri, 18 Sep 2026 05:03:48 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0
```

Final Result

The completed Patient Lookup System provides a secure full-stack solution where an authenticated user can log in, receive a JWT, access authorized patient information, search for patients, view patient details, and perform CRUD operations according to their assigned role.

The project demonstrates **Java/Spring Boot backend development, React frontend development, PostgreSQL database integration, REST APIs, JWT authentication, role-based authorization, CRUD operations, validation, responsive UI design, API documentation, and Git/GitHub version control.**